

第2章：应用层



学习目标：

- 应用层协议及实现
 - 传输层服务模型
 - 客户机/服务器模式
 - P2P模式
 - 内容分发网络
- 常见的应用层协议
 - HTTP
 - FTP
 - SMTP / POP3 / IMAP
 - DNS
- 编写网络应用程序
 - 套接字API

2-1

第2章：应用层



2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-2

常见的网络应用程序

- 电子邮件
- 网页浏览
- 远程登录
- P2P文件共享
- 多人网络游戏
- 存储视频(YouTube, Hulu, Netflix)
- IP电话 (e.g., Skype)
- 实时视频会议
- 社交网络软件
- 搜索引擎
- ...
- ...

2-3

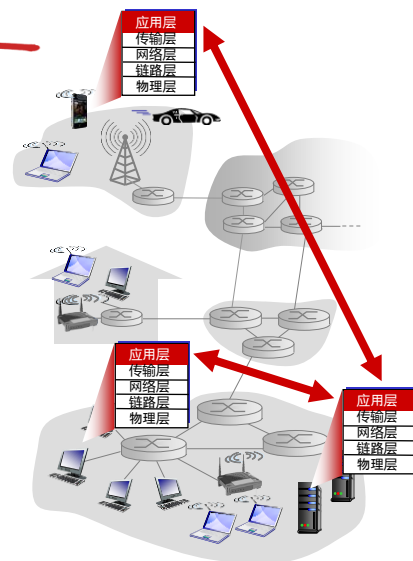
创建一个网络应用app

编写程序：

- 运行于不同端系统
- 通过网络进行通信
- e.g., web服务器软件与客户机的浏览器

不需要为通信设备编写程序

- 路由器，交换机等通常无应用层



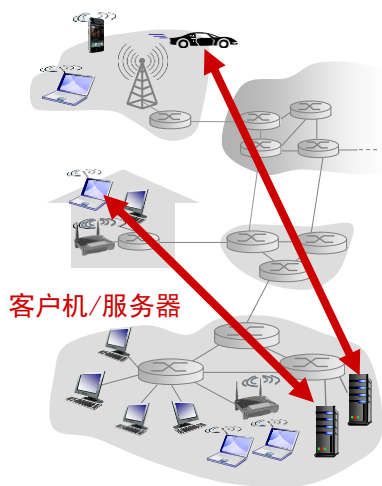
2-4

网络应用程序的体系结构

- 客户机/服务器模式
- 对等模式 (P2P)

2-5

客户机/服务器模式



服务器：

- 一直在网
- 使用固定IP地址
- 提供服务

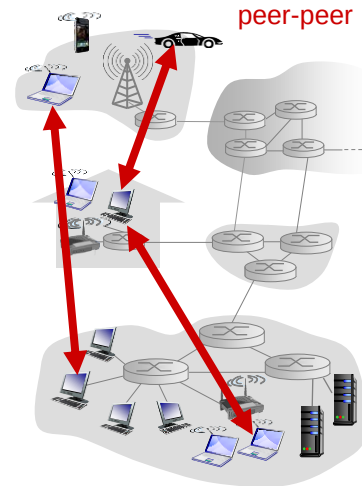
客户机：

- 可能间歇性在网
 - 可能使用动态IP地址
 - 请求服务
 - 客户机之间不通信
-
- 如web网页、电子邮件传输、文件下载等

2-6

P2P模式

- 无一直在网的服务器
- 端系统间可以通信
- 对等方请求/提供服务
 - 自扩展性- 规模和内容
- 对等方间歇性互联与更换IP地址，管理复杂
- 如BT下载，迅雷，网络视频会议



2-7

进程通信

进程: 主机中可并发执行的程序

- 主机内进程使用操作系统的进程通信机制相互通信
- 主机间进程通过交换报文相互通信

客户机，服务器

客户机进程: 发起通信

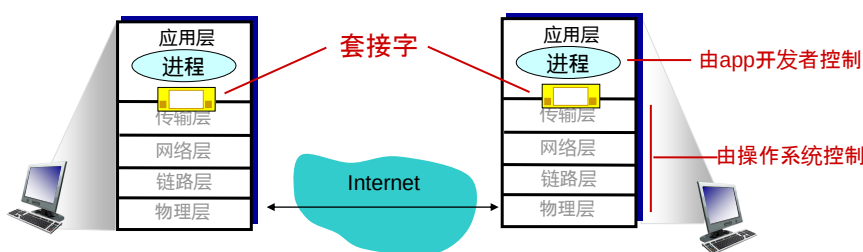
服务器进程: 等待通信请求

- 注意: P2P进程既是客户机
又是服务器

2-8

套接字Socket

- 套接字：同一台主机内应用层与运输层之间的接口。也叫应用程序和网络之间的应用程序接口API，是在网络上建立网络应用程序的可编程接口。
- 进程通过套接字从网络收发报文
- 像一扇门
 - 发送进程将报文推出门外
 - 依靠门外的传输机制传送到接收进程的套接字



2-9

进程寻址

- 主机有全网唯一的32位IP地址
- 进程使用IP地址和进程使用的端口号进行寻址
- Q: IP地址是否满足进程寻址?
No
- 端口举例:
 - web服务器进程: 80端口
 - 邮件服务器进程: 25端口
- 发送网页请求到南理工web服务器进程:
 - IP地址: 202.119.80.142
 - 端口号: 80

2-10

应用层协议的内容

- 报文种类

- 请求，响应

- 语法：

- 报文含多少字段，字段大小

- 语义

- 字段中信息的含义

- 定时规则：何时发送报文和如何进行响应

- 开放协议：

- RFC文档定义
- 允许互操作
- e.g., HTTP, SMTP

- 专有协议：

- e.g., Skype

2-11

app需要的传输层服务

- 可靠性

- 一些app要求100%可靠性 (e.g., 文件传输, web电子商务交互)
- 一些则可容忍部分出错 (e.g., 视频传输)

- 实时性

- 一些app要求低时延 (e.g., 网络电话, 交互性游戏)

- 吞吐量

- 一些app要求最小带宽 (e.g., 多媒体传输)
- 另一些无太多要求(弹性应用, 如电子邮件)

- 安全性

- 加密, 数据完整性, 身份鉴别...

2-12

常用app的传输需求

app	可靠性	吞吐量	实时性
文件传输	不能丢包	弹性	否
e-mail	不能丢包	弹性	否
网页传输	不能丢包	弹性	否
实时音视频	容忍丢包	音频: 5kbps-1Mbps 视频: 10kbps-5Mbps	是, 0.1秒
存储音视频	容忍丢包	同上	是, 几秒
交互游戏	容忍丢包	至少几kbps	是, 0.1秒
短消息交互	不能丢包	弹性	是

2-13

传输层提供的服务

TCP服务:

- **可靠传输**: 发送进程到接收进程
- **流量控制**: 控制发送进程, 使接收进程来得及接收
- **拥塞控制**: 控制发送进程, 使网络来得及传送
- **不提供**: 实时性, 最小吞吐量保证, 安全性
- **面向连接**: 需要在进程间建立连接, 通信后拆除连接

UDP服务:

- **不可靠传输**
- **不提供**: 可靠性, 流量控制, 拥塞控制, 实时性, 最小吞吐量保证, 安全性,
- **非面向连接**

Q: 为什么提供UDP?

2-14

因特网app: 应用与传输层协议

app	应用层协议	传输层协议
e-mail	SMTP [RFC 2821]	TCP
远程控制	Telnet [RFC 854]	TCP
Web网页Web	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流媒体	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
网络电话	SIP, RTP, 专有协议 (e.g., Skype)	TCP or UDP

2-15

使TCP更安全: SSL

TCP & UDP:

- 无加密
- 明文密码在网络中传输

SSL (Secure Socket Layer)

- 提供加密的TCP连接
- 数据完整性鉴别
- 身份鉴别

SSL位于应用层，有自己的套接字API

- 发送明文密码给套接字，然后在因特网加密传输
- 详见第8章

2-16



第2章：应用层

2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-17

Web服务和HTTP协议

- 网页由对象组成
- 对象：HTML文件, JPEG图片, Java applet小程序, 视频文件, ...
- 网页有一个HTML文件，以链接方式包含许多对象
- 对象使用URL(Uniform Resource Locator)统一资源定位符寻址, e.g.,

`http:// www.someSchool.edu:808 / somedepartment / picture.gif`

协议名

主机名

端口号

路径名

2-18

HTTP协议

HTTP: hypertext transfer protocol

- 应用层协议
- 客户机/服务器模式
 - **客户机**: 浏览器请求、接收(使用HTTP协议)并显示网络对象
 - **服务器**: Web服务器软件收到请求后, 发送 (使用HTTP协议) 对象



2-19

HTTP协议

使用传输层TCP服务:

1. 客户机向服务器的80 (默认) 端口发起TCP连接 (创建套接字)
2. 服务器接受连接请求, 建立TCP连接
3. 在浏览器(HTTP 客户机) 和 Web 服务器软件 (HTTP服务器)之间交换HTTP报文
4. 关闭TCP 连接

HTTP是无状态的

- 服务器不保存客户机请求信息

注意

有状态的协议是复杂的!

- 需要保存与维护过往信息
- 如果客户机/服务器崩溃, 状态信息将不一致, 需要同步

2-20

HTTP连接

非持续连接

- 在一个TCP连接上最多发送一个对象
 - 然后关闭连接
- 下载多个对象，需要多个连接

持续连接

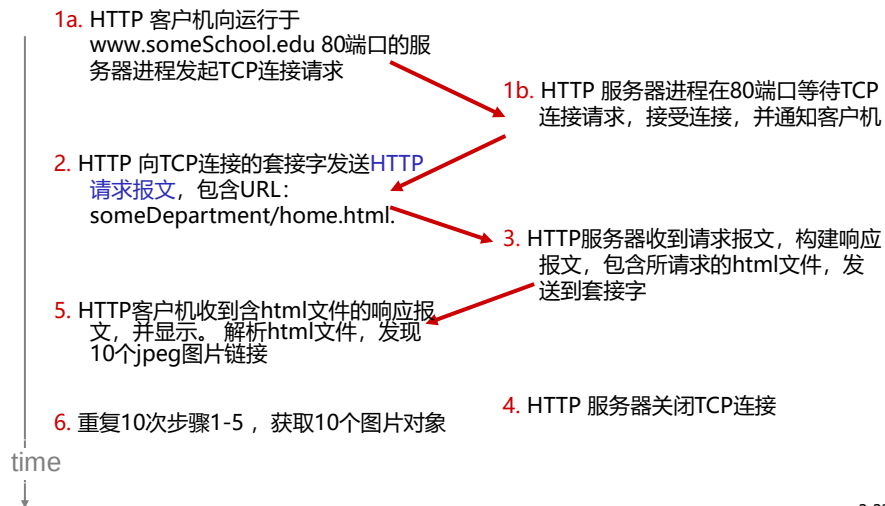
- 多个对象可以在一个TCP连接上传送

2-21

非持续连接

假设用户输入URL:

`www.someSchool.edu/someDepartment/home.html` (包含文本和10个jpeg图片链接)

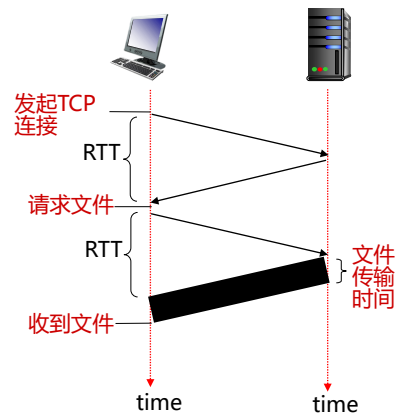


2-22

非持续连接：响应时间

往返时延RTT(Round-Trip Time)

- 非持续连接的单个对象响应时间
= 2RTT + 文件传输时间



2-23

持续连接

非持续连接:

- 每个对象至少需要2个RTT
- 浏览器在相同服务器获取不同对象，需要建立多个TCP连接
- 操作系统忙于建立与拆除TCP连接

持续连接:

- 服务器发送响应后，保持连接
- 相同服务器/客户机的后续报文使用此连接传输
- 客户机发现对象链接，就发送请求报文
- 响应时间减少

2-24

HTTP请求报文

- 两种HTTP报文: 请求报文, 响应报文
- HTTP请求报文:
 - ASCII (用户可读)

请求行(GET, POST, HEAD命令)

URL

协议版本

回车符
换行符

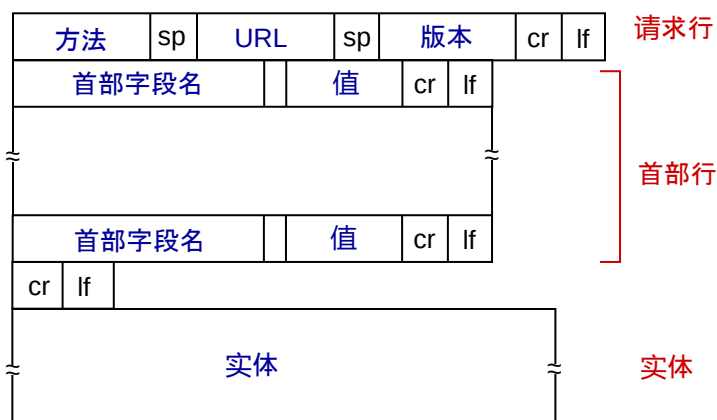
```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

首部行

实体

2-25

HTTP请求报文格式



cr: 回车(Carriage Return); If:换行(Line Feed)

2-26

上载用户表格输入

如有用户表格输入，如何上载到服务器？

POST方法:

- 存放到请求报文的实体

URL方法:

- 使用GET方法
- 存放到URL字段:

`www.somesite.com/interests?read&music`

2-27

方法

HTTP/1.0:

- GET
- POST (对象可更大、数据类型更多、更安全...)
- HEAD
 - 要求服务器返回不含对象的响应报文，用于调试

HTTP/1.1:

- GET, POST, HEAD
- PUT
 - 向URL指定位置上传实体中的对象
- DELETE
 - 删除URL指定的对象

2-28

HTTP响应报文

状态行(协议, 状态码, 状态短语)

首部行

实体,
e.g.,
所请求的
对象

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
1\r\n
\r\n
data data data data data ...
```

2-29

HTTP响应状态码

■ 状态码举例:

200 OK

- 请求成功, 对象在实体中给出

301 Moved Permanently

- 请求的对象已转移, 对象新位置在Location给出,

400 Bad Request

- 错误的请求

404 Not Found

- 请求的对象不在本服务器

505 HTTP Version Not Supported

2-30

Cookie: 记录用户状态

许多网站使用cookie

组成:

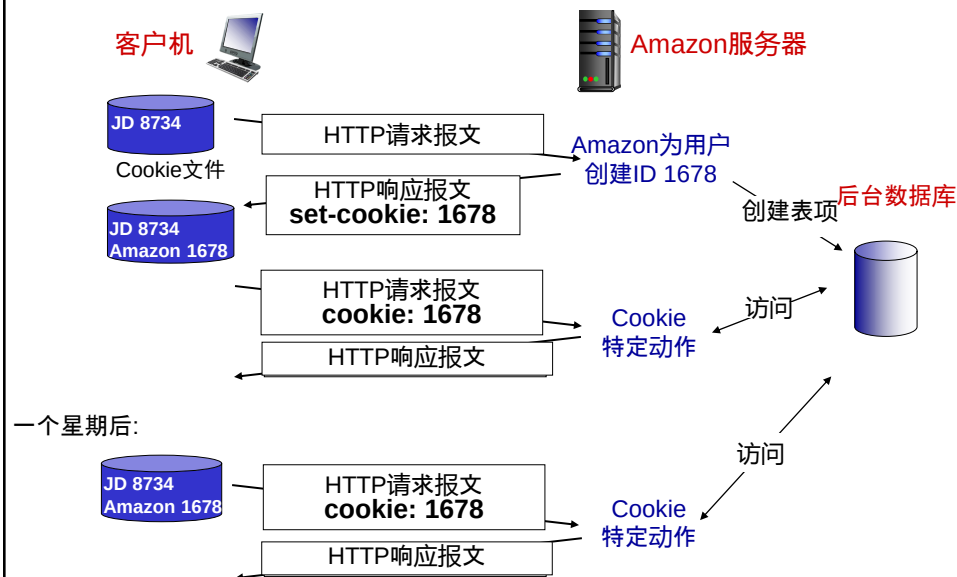
- 1) HTTP响应报文的cookie 首部行
- 2) 接下来的HTTP请求报文的cookie首部行
- 3) 客户端的cookie文件, 由浏览器管理
- 4) 服务器的后台数据库

举例:

- 张三经常上网
- 第一次访问Amazon网站
- 当HTTP请求报文到达, 网站将为张三创建:
 - 一个ID
 - 此ID的后台数据库表项

2-31

Cookie: 记录用户状态



2-32

Cookie

作用:

- 用户认证
- 购物车保存
- 商品推荐
- 保存用户信息（姓名，信用卡，地址）

注意

cookie与用户隐私:

- 网站会获取你的数据

2-33

Web缓存 (代理服务器)

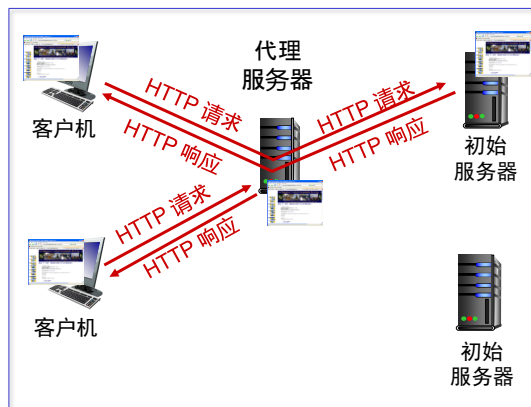
Web缓存 (Web cache): 代表初始服务器来满足HTTP请求的网络实体

初始(原始)服务器(origin server): 对象最初存放并始终保持其拷贝的服务器

目的: 无需初始服务器, 也能响应客户HTTP请求

流程:

- 用户设置浏览器, 通过代理服务器访问网络 (给出其IP地址和端口号)
- 浏览器将所有的HTTP请求发给代理服务器
 - 如果对象在代理服务器中, 直接返回
 - 否则代理服务器向原始服务器发送请求, 将响应返回给客户机, 同时缓存对象



2-34

代理服务器

- 既是客户机，
也是服务器
 - 通常由大学，公司等
机构ISP设置
- 作用：
- 减少响应时间
 - 减少机构的接入通信量
 - 内容提供商更高效地提供服务

2-35

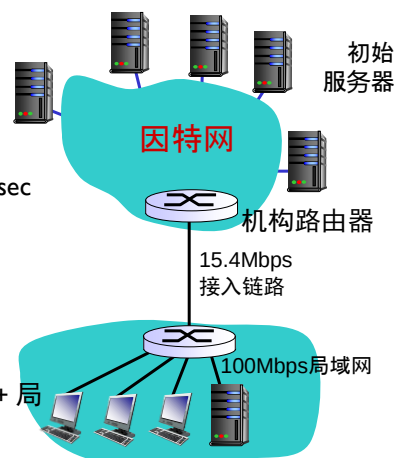
代理服务器例子

假设：

- 传输对象平均大小: 1M bits
- 局域网浏览器发送请求频率: 15个/sec
- 因此请求平均速率: 15 Mbps
- 从机构路由器到初始服务器的RTT: 2 sec
- 接入链路速率: 15.4Mbps

结果：

- 局域网流量强度: 15%
- 接入链路流量强度= 99% *problem!*
- 上网总时延= 因特网时延+ 接入时延+ 局域网时延
= 2 sec + 若干分钟 + 若干毫秒



2-36

代理服务器例子: 接入链路提速

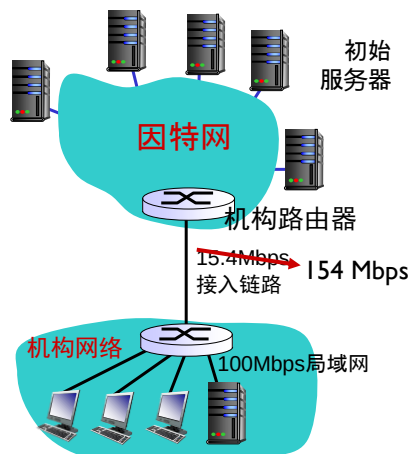
假设:

- 对象平均大小: 1M bits
- 所有浏览器发送请求: 15个/sec
- 请求的平均速率: 15 Mbps
- 从机构路由器到初始服务器的RTT: 2 sec
- 接入链路速率: ~~15.4 Mbps~~ → 154 Mbps

结果:

- 局域网流量强度: 15%
- 接入链路流量强度 = ~~99%~~ → 9.9%
- 总时延 = 因特网时延 + 接入时延 + 局域网时延
= 2 sec + ~~若干分钟~~ + 若干毫秒
 若干毫秒

代价: 接入链路提速10倍 (贵!)



2-37

代理服务器例子: 使用代理服务器

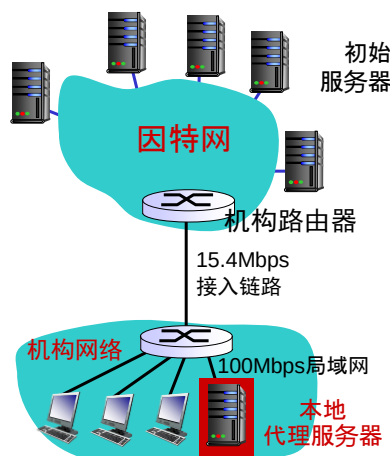
假设:

- 对象平均大小: 1M bits
- 所有浏览器发送请求: 15个/sec
- 请求的平均速率: 15 Mbps
- 从机构路由器到初始服务器的RTT: 2 sec
- 接入链路速率: 15.4 Mbps

结果:

- 局域网流量强度: 15%
- 接入链路流量强度 = ?
- 总时延 = ?

代价: 设置本地代理服务器 (划算!)



2-38

代理服务器例子: 使用代理服务器

计算接入流量强度和总时延:

- 假设代理服务器的命中率为0.4

- 60% 的请求使用接入链路

- 流入接入链路的数据速率 =

$$0.6 * 15 \text{ Mbps} = 9 \text{ Mbps}$$

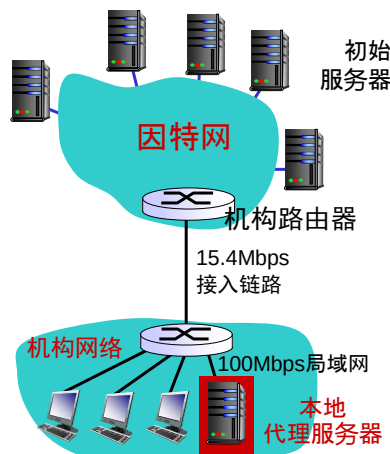
- 接入链路流量强度 = $9 / 15.4 = 58\%$

- 总时延

- = $0.6 * \text{与初始服务器通信时延} + 0.4 * \text{与代理服务器通信时延}$

$$= 0.6 * 2.01 + 0.4 * \text{若干毫秒} = \sim 1.2 \text{ 秒}$$

- 比给接入链路提速还少, 并且还便宜!



2-39

条件GET方法

代理服务器



初始服务器



- 作用:** 代理服务器检查缓存对象是否过时

- 代理服务器:** 在HTTP请求中给出缓存对象的日期

If-modified-since:
<date>

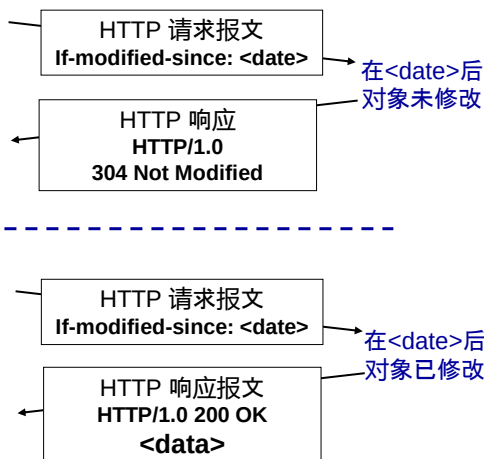
- 初始服务器:**

- 如果对象未修改, 则返回不含对象的响应:

HTTP/1.0 304 Not
Modified

- 如果对象已修改, 则返回包含对象的响应:

HTTP/1.0 200 OK
<data>



2-40



第2章：应用层

2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-41

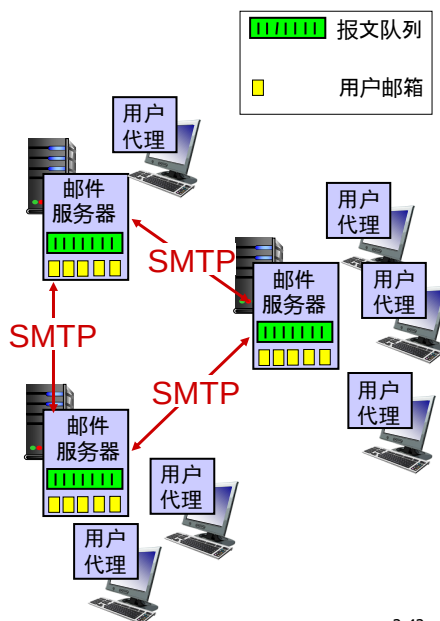
电子邮件

组成：

- 用户代理
- 邮件服务器
- SMTP (simple mail transfer protocol)

用户代理

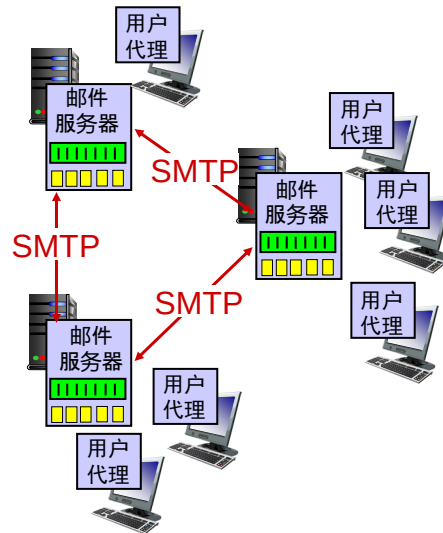
- 是用户与网络应用程序之间的接口
- 撰写，编辑，读取邮件
- e.g., Outlook, Foxmail, iPhone邮件客户端
- 向/从邮件服务器发送/接收报文
- Web应用的用户代理是？



2-42

邮件服务器

- **邮箱**为用户存放收到的邮件
- **报文队列**为用户存放要发送的邮件
- **SMTP 协议**
 - 客户机
 - 服务器



2-43

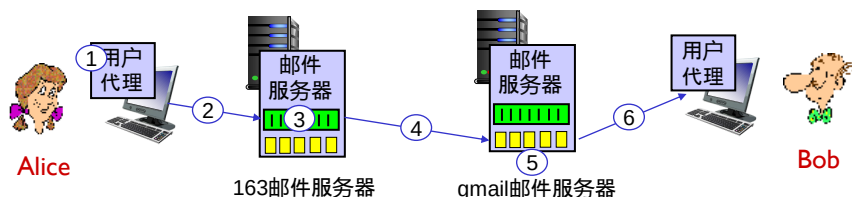
SMTP [RFC 2821]

- 使用TCP传输邮件到服务器25端口
- 三个阶段
 - 握手
 - 传输报文
 - 关闭
- 命令/响应交互(类似于HTTP)
 - **命令**: ASCII文本
 - **响应**: 状态码和状态短语
- 使用7位ASCII,对包含非7位ASCII字符的报文或二进制数据(如图片、声音等), 需按照7位ASCII码进行编码 (MIME编码)
 - **MIME(Multipurpose Internet Mail Extensions)**: 多用途因特网邮件扩展, 用于传输非ASCII数据

2-44

举例

- 1) Alice (Alice@163.com)使用用户代理撰写邮件给 bob@gmail.com,
- 2) 用户代理发送邮件到她的163邮件服务器, 并进入队列排队等待转发
- 3) 163邮件服务器作为SMTP客户端打开与 gmail邮件服务器的TCP连接
- 4) 163邮件服务器在TCP连接上发送邮件到gmail邮件服务器
- 5) Gmail服务器将邮件放入Bob的邮箱
- 6) Bob使用他的用户代理获取并阅读邮件



2-45

举例：SMTP交互（邮件服务器之间）

```
S: 220 gmail.com
C: HELO 163.com
S: 250 Hello 163.com, pleased to meet you
C: MAIL FROM: <alice@163.com >
S: 250 alice@163.com... Sender ok
C: RCPT TO: <bob@gmail.com >
S: 250 bob@ gmail.com ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 gmail.com closing connection
```

2-46

SMTP

- SMTP 使用持续连接
- SMTP使用7位ASCII码
- SMTP 使用CRLF.CRLF来判断邮件的束

与HTTP的比较:

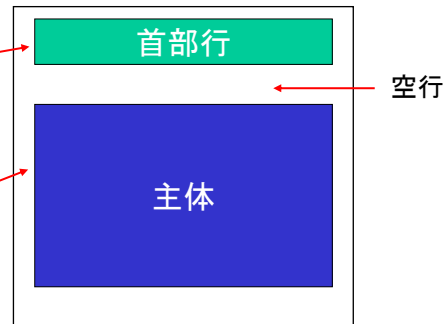
- HTTP: 拉
- SMTP: 推
- 都使用ASCII的命令与响应
- HTTP: 一个对象封装在一个报文中
- SMTP: 多个对象在一个报文中（结合MIME,包含非ASCII对象，如图片，附件等）

2-47

邮件格式

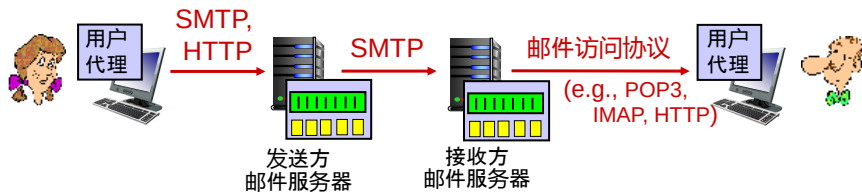
RFC 5322: 定义ASCII文本邮件格式:

- 首部行, e.g.,
 - To:
 - From:
 - Subject:
- 主体: 用户邮件内容
 - 只能是ASCII字符
 - 可结合MIME, 包含非ASCII对象



2-48

邮件访问协议



- 用户代理从邮件服务器读取邮件
 - **POP3**: Post Office Protocol Version 3 [RFC 1939]: 用户验证, 邮件下载后操作
 - **IMAP**: Internet Mail Access Protocol [RFC 1730]: 更多功能, 包括在服务器上的邮件操作
 - **HTTP**: 使用浏览器收发邮件, 普遍使用

2-49

POP3协议

验证阶段

- 客户机命令:
 - **user**
 - **pass**
- 服务器响应:
 - **+OK**
 - **-ERR**

事务阶段

- 客户机命令:
 - **list**: 给出邮件数量
 - **retr**: 按标号获取邮件
 - **dele**: 按标号删除邮件
 - **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

2-50

POP3和IMAP

POP3

- 上面例子中，POP3使用了“下载并删除”模式
 - Bob如果换了电脑，无法再获取邮件
- 而POP3的“下载并保持”模式：在不同客户端都能下载邮件
- POP3是无状态的

IMAP

- 邮件只存在于服务器
- 允许邮件放入不同文件夹
- 记录用户状态

2-51

第2章：应用层



2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-52

DNS: 域名系统

人: 多种识别符:

- 姓名, 身份证号, 护照号

因特网主机、路由器:

- IP 地址(32 或 128 bit) : 网络和设备使用
- 域名, e.g.,
www.njust.edu.cn :
人使用

域名系统DNS:

- 分布式数据库: 许多域名服务器, 采用层次结构
- DNS协议: 主机与域名服务器通信, 实现名字解析

Q: IP地址和域名如何相互映射

2-53

DNS: 作用与结构

作用:

- 域名到IP地址的转换
- 主机别名
- 邮件服务器别名
- 负载分配
 - web服务器集群: 一个域名对应多个IP地址与服务器

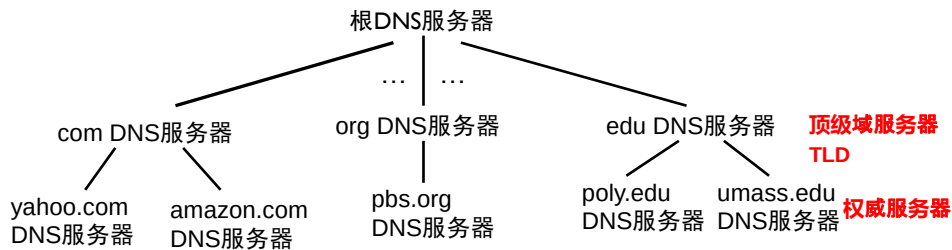
为何不采用集中式DNS?

- 单点故障
- 通信拥塞
- 距离远则吞吐量小
- 维护频繁(IP变动)

A: 无法扩展!

2-54

DNS: 分布式、层次式数据库



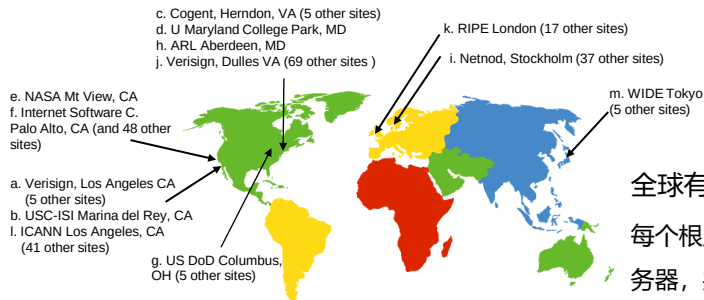
客户机解析www.amazon.com的IP地址:

- 客户机请求根服务器找到com DNS 服务器
- 客户机请求com DNS服务器找到amazon.com DNS服务器
- 客户机请求amazon.com DNS服务器解析www.amazon.com的IP 地址

2-55

DNS: 根域名服务器

- 本地域名服务器不能解析，则联系根服务器
- 根域名服务器:
 - 联系顶级域服务器/权威服务器
 - 获取地址映射
 - 将结果返回本地域名服务器



全球有13个根服务器

每个根服务器有多个备份服务器，共400多个

2-56

顶级域服务器，权威服务器

顶级域服务器TLD（一级域）：

- 负责com, org, net, edu, aero, jobs, museums, 和所有顶级国家或地区域, e.g.: cn, uk, fr, ca, jp
- Verisign公司维护com 顶级域服务器
- Educause公司维护edu顶级域服务器

权威服务器:

- 机构的域名服务器，提供本机构主机的地址映射服务
- 由机构或ISP维护

2-57

本地域名服务器

- 不属于层次结构
- 每个ISP 有一个，又称：默认域名服务器
- 主机的DNS 查询，先被送到默认域名服务器
 - 缓存了近期的域名解析结果（有可能过期哦）
 - 未命中，则将查询转发到层次结构

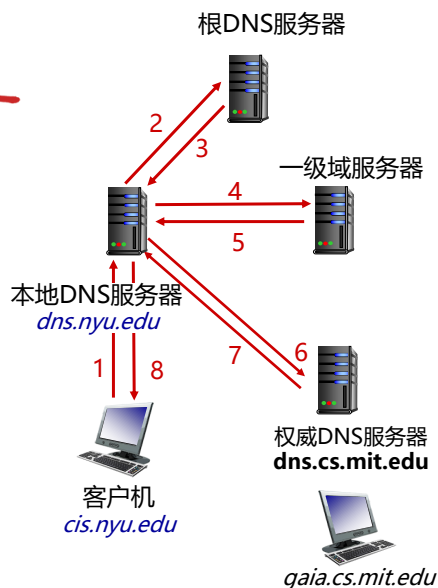
2-58

DNS域名解析例子

- 位于cis.nyu.edu 的主机想解析gaia.cs.mit.edu的IP地址

迭代查询 (Iterative) :

- 层次结构服务器如果不能解析，将返回其他服务器地址
- “我无法解析，你可以去问一下这个服务器”

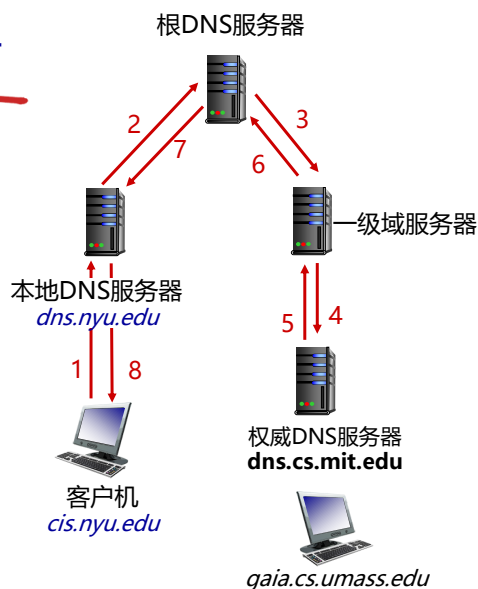


2-59

DNS域名解析例子

递归查询 (Recursive) :

- 层次服务器承担解析责任
- 增加了上层服务器任务



2-60

DNS: 缓存，更新记录

- 域名服务器缓存解析结果
 - 通常，一级域服务器的映射信息也会缓存到本地域服务器
 - 因此，根服务器不经常被访问
- 缓存记录可能过时
 - TTL超时后，缓存记录被删除
 - 例如主机IP地址的改变，直到超时后，才会被知道

2-61

DNS 记录

DNS记录: (name, value, type, TTL)

type=A

- name 是域名
- value 是IP地址

type=NS

- name 是域(e.g., njust.com)
- value 是域中权威域名服务器的地址

type=CNAME

- name 是主机别名
例如: www.ibm.com是 servereast.backup2.ibm.com的别名
- value 是规范主机名

type=MX

- value 是name主机的邮件服务器的域名

2-62

DNS报文

- **请求和应答**报文, 使用相同格式

首部 (12字节) :

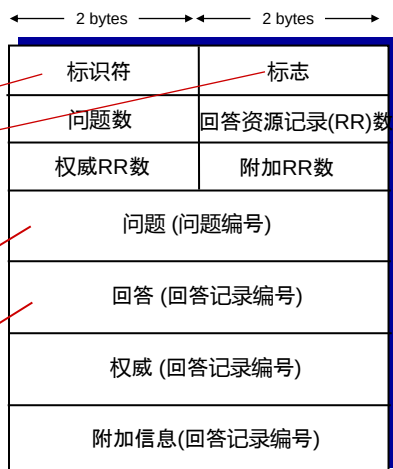
- **标识符**: 16 bit, 应答与请求报文使用相同标识符

- **标志**:

- 请求或应答
- 希望递归查询
- 递归查询可用
- 回答是权威的

name, type

RRs



2-63

如何新增DNS记录

- 例子: 新创企业 "Network Utopia"
- 在注册登记机构注册域名networkutopia.com, 提供企业权威服务器的域名和IP地址
 - 注册登记机构插入两条记录到.com 一级域服务器:
(networkutopia.com, dns.networkutopia.com, NS)
(dns.networkutopia.com, 212.212.212.1, A)
- 在权威服务器中创建两条记录:
www.networkutopia.com的type A记录
networkutopia.com邮件服务器的type MX记录

2-64

DNS攻击

DDoS攻击

- 攻击根服务器
 - 分组过滤
 - 多数本地服务器缓存了一级域服务器地址，无需根服务器
 - 损害很小
- 攻击一级域服务器
 - 更为有效

重定向攻击

- 截获DNS请求报文
- 发送假的DNS应答报文，并被域名服务器缓存

2-65

第2章：应用层



2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

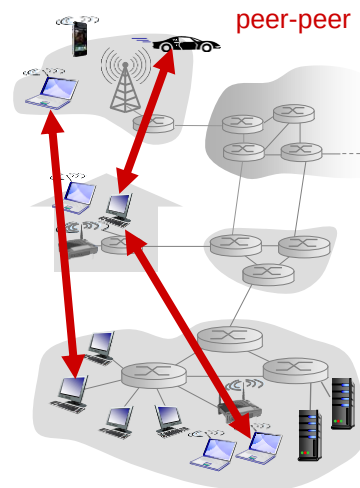
2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-66

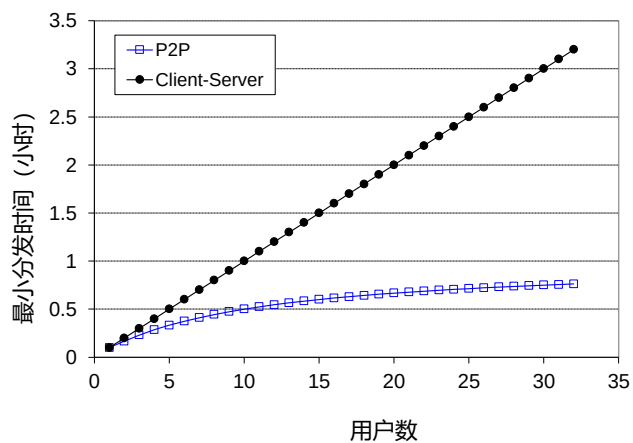
P2P模式

- 无需一直在网的服务器
- 端系统间可以通信
- 对等方间歇性互联、更换IP地址，管理复杂
- 如BT下载，迅雷，网络视频会议



2-67

C/S vs. P2P



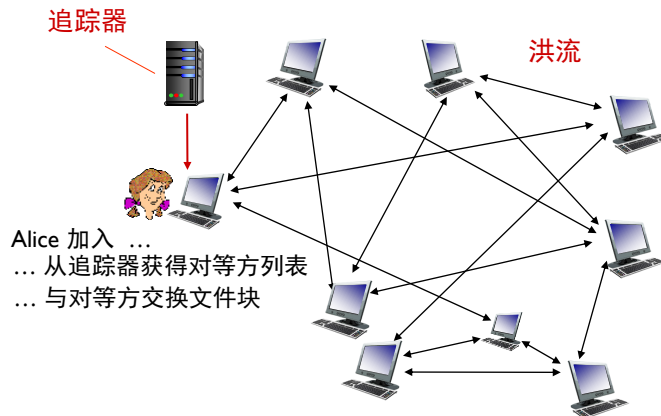
2-68

P2P文件分发: BitTorrent

洪流 (Torrent) : 参与一个特定文件分发的对等方(Peer)的集合

追踪器 (tracker) : 跟踪洪流中对等方的实体

- 文件切分成256KB的块
- 洪流中的对等方相互发送和接收文件块

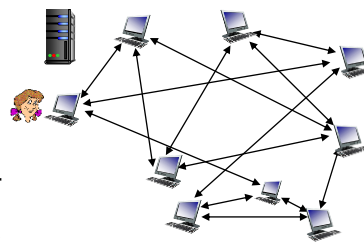


2-69

P2P文件分发: BitTorrent

▪ 对等方加入洪流:

- 没有文件块, 然后累积
- 在追踪器注册, 获得对等方随机子集(邻居)列表, 并与其建立TCP连接



- 下载的同时, 上载
- 对等方可随时加入和离开洪流, 邻居也是变动的
- 下载完成后, 可以离开或无私地留在洪流中

2-70

BitTorrent: 请求与发送策略

请求块:

- 不同对等方有不同的文件块
- Alice周期性询问邻居有什么
- Alice请求自己没有的块, 采用**最稀缺优先策略**

发送块: 一报还一报策略

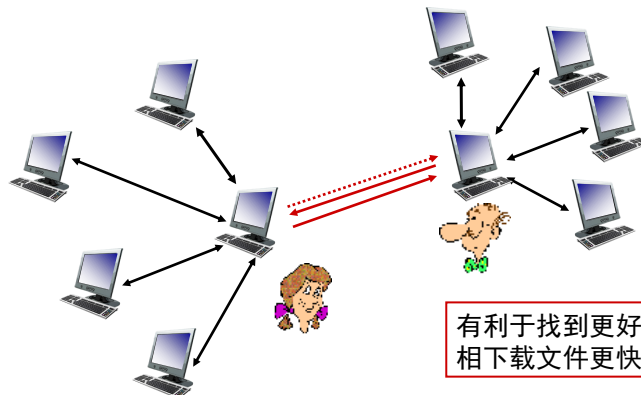
(*tit-for-tat*)

- Alice只发送给向她发送速率最快的4个对等方
 - 每10秒钟评估一下, 前4名
- 每30秒: 随机选择一个对等方, 发送文件块
 - 此等对方也可能进入前4

2-71

BitTorrent: tit-for-tat

- (1) Alice 随机选择了 Bob并发送
- (2) Alice 成为Bob的前4名提供者; Bob 进行报答
- (3) Bob成为Alice的前4名提供者



2-72



第2章：应用层

2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

2.7 使用UDP和TCP的套接字编程

2-73

视频流和内容分发网

- 视频流：因特网的流量主体
 - YouTube、Netflix: 37%, 16% 住宅ISP流量
- 挑战1: 如何支持不同速率用户?
- 挑战2: 如何支持10亿级别用户?
 - 单一服务器或数据中心，不行(why? Later)

YouTube

NETFLIX

hulu



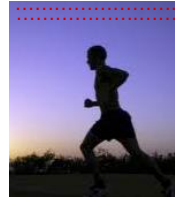
✓ 解决方案：DASH、CDN

2-74

视频

- 视频: 一定速率播放的图片
 - e.g., 24 帧/秒
- 数字图片: 像素点阵
 - 每个像素用若干比特表示
- 编码: 利用图片内或图片间的冗余, 减少比特数
 - 空间冗余
 - 时序冗余

空间编码例子: 无需发送N个相同的紫色像素, 只需发送两个值: 紫色和重复的次数



第 i 帧

时序编码例子: 无需发送整个第 $i+1$ 帧, 只需发送与前一帧不同的地方



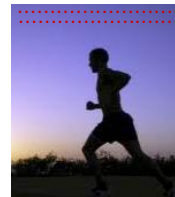
第 $i+1$ 帧

2-75

视频

- 固定速率编码
- 可变速率编码
- 示例:
 - MPEG I (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (通常用于因特网, < 1 Mbps)

空间编码例子: 无需发送N个相同的紫色像素, 只需发送两个值: 紫色和重复的次数



第 i 帧

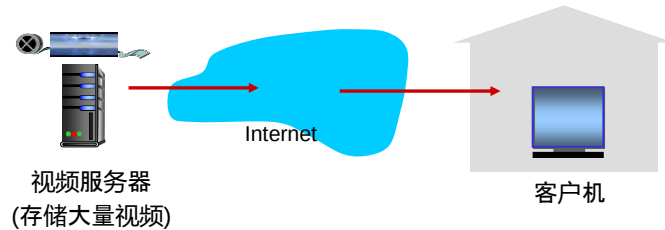
时序编码例子: 无需发送整个第 $i+1$ 帧, 只需发送与前一帧不同的地方



第 $i+1$ 帧

2-76

存储视频流



2-77

多媒体流传输: DASH

- **DASH: Dynamic, Adaptive Streaming over HTTP** 基于HTTP的动态自适应视频流
- **服务器:**
 - 每个视频有多个版本, 以不同速率编码并存储
 - 每个版本切分成若干块 (几秒长度)
 - 告示文件 **manifest file**: 提供不同版本的URL及其比特率
- **客户机:**
 - 周期性评估服务器到客户机的带宽
 - 查询告示文件, 一次请求一个块
 - 请求当前带宽支持的最大编码速率的块
 - 依据带宽变化, 不同时间请求不同编码速率的块

2-78

内容分发网CDN

- **挑战:** 如何从百万视频中找到特定视频，并传送给数以万计的并发用户？
 - **方案 1:** 单个大的服务器
 - 单点故障
 - 网络拥塞
 - 长距离客户速率低
 - 热门视频在链路上的多次传送
- **无法规模增长**

2-79

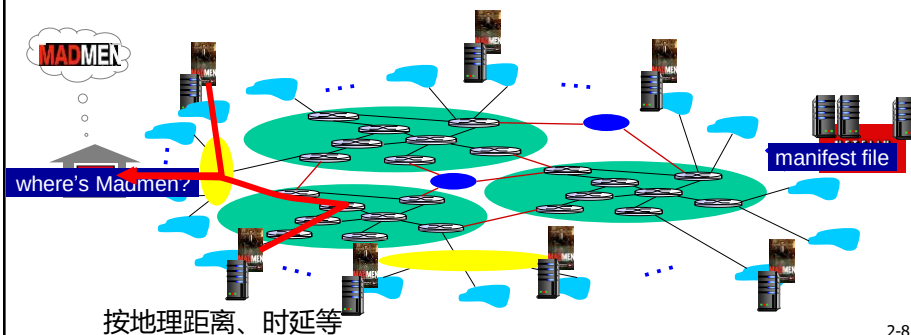
内容分发网CDN

- **方案 2:** 将视频多个拷贝存储在不同的地点 (CDN)，分为专用CDN(如Google的CDN，只运行自己的Youtube)和第三方CDN(如Akamai)
- 两种服务器安置原则:**
- **深入:** 将CDN 服务器深入部署到接入网
 - 更靠近用户
 - Akamai使用, 1700 CDN节点
 - 优点: 改善用户延时和吞吐量; 缺点: 维护和管理难
 - **邀请做客:** 通过少量 (如10个) 关键位置建造大集群来邀请到ISP做客，不是将集群放在接入网，通常放在IXP
 - Limelight和许多其他CDN公司采用
 - 优点: 较低的维护和管理开销; 缺点: 用户的较高延时和较低吞吐量

2-80

内容分发网CDN

- CDN: 将内容拷贝存储到不同CDN节点
 - e.g., Netflix存储MadMen电影拷贝
- 用户从CDN请求内容
 - 将用户导向附近的拷贝, 并下载
 - 如果拥塞, 可能选择不同的拷贝

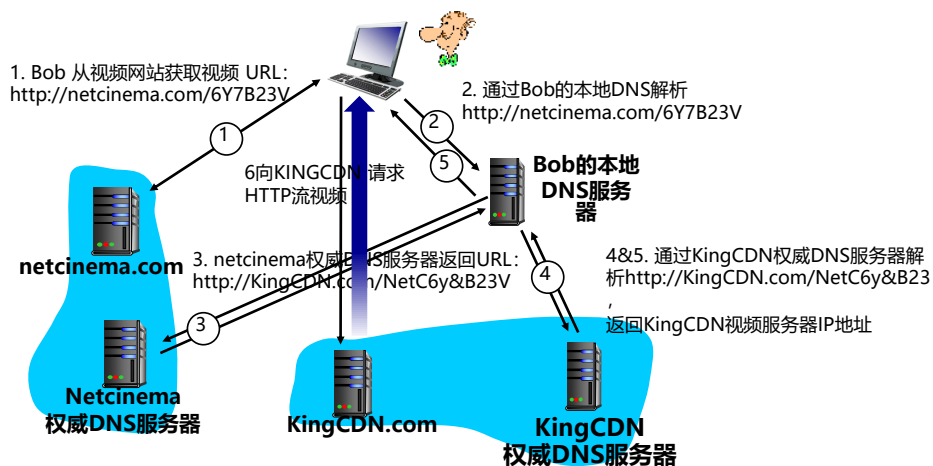


2-81

内容分发网CDN：具体流程

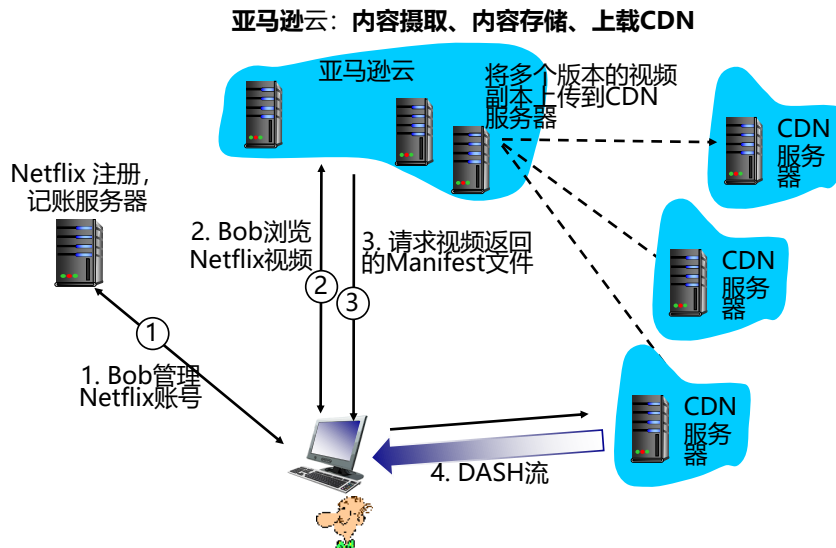
Bob 请求视频 <http://netcinema.com/6Y7B23V>

- 视频存储在CDN: <http://KingCDN.com/NetC6y&B23V>



2-82

内容分发网CDN：Netflix案例



2-83

第2章：应用层



2.1 网络应用程序工作原理

2.2 Web 和 HTTP

2.3 电子邮件

- SMTP, POP3, IMAP

2.4 DNS

2.5 P2P 应用

2.6 视频流和内容分发网

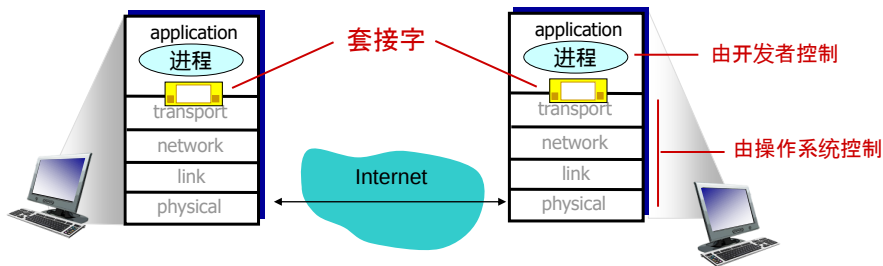
2.7 使用UDP和TCP的套接字编程

2-84

套接字编程

目标: 构建客户机/服务器应用程序

套接字: 应用层进程与端到端传输层协议之间的一扇门



2-85

套接字编程

两种不同的传输层协议:

- **UDP:** 不可靠, 高效数据报
- **TCP:** 可靠, 面向比特流

套接字编程例子:

1. 客户机从键盘读取一行字符, 并发给服务器
2. 服务器收到后, 转换成大写字符
3. 服务器将转换后字符发给客户机
4. 客户机收到后, 显示到屏幕上

2-86

UDP套接字编程

UDP: C/S之间无连接

- 无握手过程
- 发送方将接收方IP地址和端口号加入每个分组
- 接收方从收到的分组获知发送方的IP地址和端口号

UDP: 分组可能丢失或失序

应用程序角度:

UDP提供C/S之间的高效、不可靠数据报传输

2-87

UDP套接字交互

服务器 (运行于serverIP, 端口x)

创建套接字:
`serverSocket =
socket(AF_INET, SOCK_DGRAM)`

从`serverSocket`
读取数据报

构建含客户机地址
和端口号的响应数据报;
通过`serverSocket`发送

客户机

创建套接字:
`clientSocket =
socket(AF_INET, SOCK_DGRAM)`

构建含server IP和port=x
的数据报; 通过`clientSocket`
发送数据报

从`clientSocket`
读取数据报

关闭
`clientSocket`

2-88

Python示例: UDP客户机

```
包含Python的套接字库 → from socket import *
                           serverName = 'hostname'
                           serverPort = 12000
创建服务器的UDP套接字 → clientSocket = socket(AF_INET, SOCK_DGRAM)

获取用户键盘输入 → message = raw_input("Input lowercase sentence:")
加入服务器地址和端口号; 一起通过套接字发送 → clientSocket.sendto(message.encode(),
                                                (serverName, serverPort))

读取服务器回复信息 → modifiedMessage = clientSocket.recvfrom(2048)
打印收到的字符, 关套接字 → print modifiedMessage.decode()
                           clientSocket.close()
```

2-89

Python示例: UDP服务器

```
                           from socket import *
                           serverPort = 12000
创建UDP套接字 → serverSocket = socket(AF_INET, SOCK_DGRAM)
绑定套接字到本地端口12000 → serverSocket.bind(("", serverPort))
                           print ("The server is ready to receive")

无限循环 → while True:
从UDP套接字获取信息、客户的IP地址和端口号 → message, clientAddress = serverSocket.recvfrom(2048)
                           modifiedMessage = message.decode().upper()
发送大写字母给客户机 → serverSocket.sendto(modifiedMessage.encode(),
                                                clientAddress)
```

2-90

TCP套接字编程

客户机联系服务器

- 服务器进程必须先运行
- 服务器进程必须创建**欢迎套接字**等待客户机联系

客户机如何联系服务器:

- 创建TCP套接字, 给出服务器进程的IP地址和端口号
- 然后C/S在传输层进行三次握手 (对应用层程序透明), 建立TCP连接

- 服务器收到连接请求后, 在新端口上创建一个**连接套接字**用于与此客户进程通信
 - 使得服务器可同时与多个客户机通信
 - 使用源端口号区分不同客户进程

应用程序角度:

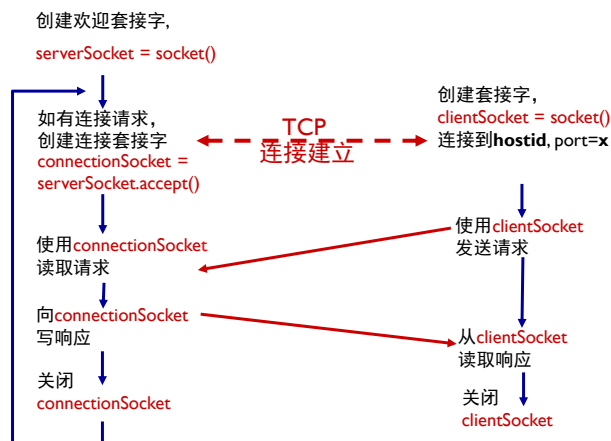
TCP提供可靠的字节流
(管道) 传输

2-91

C/S套接字交互: TCP

服务器 (运行于`hostid`, `port=x`)

客户机



2-92

Python示例:TCP客户机

```
from socket import *
serverName = 'servername'
serverPort = 12000
创建TCP客户机套接字 → clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = raw_input('Input lowercase sentence:')
报文段无需服务器IP地址和端口号 → clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

2-93

Python示例:TCP服务器

```
from socket import *
serverPort = 12000
创建TCP欢迎套接字 → serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
服务器开始侦听客户机的TCP连接请求 → serverSocket.listen(1)
print 'The server is ready to receive'
无限循环 → while True:
服务器进程等待客户机请求, 如有则创建连接套接字 → connectionSocket, addr = serverSocket.accept()
读取数据信息(不含IP地址和端口号) → sentence = connectionSocket.recv(1024).decode()
capitalizedSentence = sentence.upper()
connectionSocket.send(capitalizedSentence.encode())
关闭与此客户的连接(欢迎套接字仍在侦听) → connectionSocket.close()
```

2-94

补充习题

1. 简述应用程序体系结构两种类型的特点。
2. 什么是套接字、用户代理和Web缓存？简述用户进程和套接字的关系。

书上第2章：复习题3、5、16、习题1